

CampusEvents

College Event Management Portal

Complete Software Engineering Project Documentation

Feasibility Study • SRS • System Design • Incremental Development • Testing • Deployment • Presentation

Developed using the Incremental SDLC Model
Codebase designed and reviewed against SOLID Principles

Prepared by: Humza
Amity University, Kolkata — B.Tech Computer Science (Batch 2023–2027)

Document Version: 1.0
Date: 27 August 2026

Table of Contents

Phase 0 — Feasibility Study & Project Charter	3
Phase 1 — Software Requirements Specification (SRS)	7
Phase 2 — System Design	12
Phase 3 — Increment 1: Auth, Browsing, Registration, Creation	19
Phase 4 — Increment 2: Approval Workflow & Notifications	34
Phase 5 — Increment 3: Attendance (QR Check-in) & Certificates.....	41
Phase 6 — Increment 4: Admin Analytics & Feedback	47
Phase 7 — Overall Testing Strategy.....	52
Phase 8 — Deployment Plan	54
Phase 9 — Presentation Summary	56

Note: Page numbers correspond to this document's current pagination. If content is edited, page numbers may shift.

PHASE 0 — FEASIBILITY STUDY & PROJECT CHARTER

0.1 Introduction

This document is the single, consolidated engineering record for CampusEvents, a College Event Management Portal built end-to-end following the Incremental Software Development Life Cycle model. It contains every phase of the process — feasibility, requirements, design, four development increments (each with its own mini-requirements, design, SOLID-compliant TypeScript implementation, and test cases), overall testing strategy, deployment plan, and a presentation summary — in one continuous, traceable document.

CampusEvents replaces the manual, fragmented processes colleges currently use to announce, approve, register for, and manage events (technical fests, cultural events, workshops, seminars, and club activities) with a single web-based portal serving four distinct user roles.

0.2 Problem Statement

In the absence of a centralized system, colleges running many events per semester across clubs and departments face recurring, well-documented problems:

- Event announcements scattered across WhatsApp groups, notice boards, and individual social media handles, with no single source of truth.
- Manual, paper- or spreadsheet-based registration that does not scale and is error-prone.
- No structured approval workflow — organizers seek faculty/administrative sign-off informally, with no audit trail.
- No reliable way to verify attendance, issue certificates, or measure event outcomes.
- Administration has no aggregated view of event activity, participation trends, or club performance across the institution.

These five problems map directly onto the four user roles and the four-increment plan defined later in this document.

0.3 Feasibility Study

A feasibility study precedes requirements and design to establish that the project is worth pursuing and correctly scoped, assessed across four standard dimensions.

0.3.1 Technical Feasibility

Layer	Technology	Justification
Frontend	HTML + CSS + Vanilla JS	No build tooling required; talks to backend only via <code>fetch()</code> ; keeps the client thin and framework-free.
Backend	Node.js + TypeScript	Strong typing reduces defects in a multi-role authorization system; async I/O suits a request-heavy portal.
Database	Supabase (Postgres)	Managed Postgres with built-in Auth and Row-Level Security (RLS), removing the need to hand-roll auth infrastructure.

Layer	Technology	Justification
Hosting	Vercel (frontend) + Render (backend API) + Supabase (DB)	All three offer free/student tiers sufficient for academic-scale deployment.
Auth	Supabase Auth (JWT-based)	Provides email/password and role claims out of the box; integrates directly with Postgres RLS.

Conclusion: The project is technically feasible — all required capabilities are directly supported by the chosen stack without custom infrastructure.

0.3.2 Economic Feasibility

Resource	Free Tier Limit	Expected Load	Verdict
Supabase (DB + Auth)	500 MB DB, 50K MAUs	Single college, low thousands of rows	Comfortably within limits
Render (backend)	Free web service (sleeps on idle)	Low, bursty traffic around events	Acceptable; cold-start documented
Vercel (frontend)	Generous free bandwidth	Static assets, low bandwidth	Comfortably within limits
Domain / SSL	Not required (subdomain)	N/A	\$0 cost

Conclusion: Total monetary cost of build and deployment is \$0.

0.3.3 Operational Feasibility

- Students already use web/mobile apps for registration-style tasks — the learning curve is low.
- Organizers need only a simple event-creation form; no specialized training required.
- Faculty coordinators need a single approve/reject action with visible justification — a familiar pattern.
- Super Admins get a read-mostly analytics view, requiring no new operational process.

Conclusion: The system fits naturally into existing user workflows and is operationally feasible.

0.3.4 Schedule Feasibility

Phase	Duration	Deliverable
Phase 0 — Feasibility & Charter	Week 1	This section
Phase 1 — Requirements (SRS)	Week 2	SRS, use cases
Phase 2 — System Design	Weeks 2–3	Architecture, ER diagram, DFDs, API contracts, wireframes
Increment 1 — Auth, Browsing, Registration, Creation	Weeks 3–5	Working MVP, tests, demo
Increment 2 — Approval Workflow, Notifications	Weeks 6–7	Working increment, updated tests

Phase	Duration	Deliverable
Increment 3 — Attendance / QR Check-in, Certificates	Weeks 8–9	Working increment, updated tests
Increment 4 — Admin Analytics, Feedback	Weeks 10–11	Working increment, updated tests
Final Testing, Deployment & Presentation	Weeks 12–13	Deployed system, report, slide deck

0.4 Why the Incremental Model

Waterfall was rejected because approval-workflow and notification requirements, while understood at a high level, are likely to be refined once real organizers and coordinators use a working system — and late change under Waterfall is disproportionately expensive. Pure Prototyping was rejected because the system's architectural complexity (multi-role auth, RLS-backed access control, workflow state machines) is too substantial to de-risk with a disposable prototype alone. Spiral was rejected as disproportionate — this is not the high-cost-of-failure, high-uncertainty class of system the Spiral model targets.

The Incremental model fits because the product decomposes cleanly into functional groups that each deliver real user value (browse/register → organize → approve/notify → certify → analyze), early increments are independently demonstrable, each increment is fully tested before the next is layered on, and the overall architecture and data model are still planned up front — avoiding the integration drift that unplanned incremental work can produce.

0.5 Project Scope

In Scope

- Role-based authentication for four roles: Student, Club/Organizer, Faculty Coordinator, Super Admin.
- Event lifecycle: creation → approval/rejection → publishing → registration → attendance → completion.
- Student-facing event discovery, filtering, and registration.
- Organizer-facing event creation and management dashboard.
- Faculty Coordinator approval workflow with audit trail.
- Attendance tracking via QR-code check-in.
- Auto-generated participation certificates (PDF).
- Super Admin analytics dashboard (participation trends, club activity, approval turnaround).
- Email/in-app notifications for key state transitions.

Out of Scope (Explicitly Deferred)

- Payment processing for paid events.
- Native mobile applications.
- Multi-institution / multi-tenant support.
- Real-time chat or discussion forums.

0.6 Stakeholders

Stakeholder	Role	Primary Interest
Students	End users	Easy discovery and registration for events
Club / Event Organizers	End users	Simple event creation and participant management
Faculty Coordinators	End users / approvers	Oversight and control over what gets published
College Administration	Sponsor / Super Admin	Institution-wide visibility and reporting
Developer (Humza)	Sole developer, architect, tester	Deliver a working, gradeable, portfolio-quality system
Course Faculty / Evaluator	Academic assessor	Evidence of correct SDLC process, not just working code

0.7 High-Level Risk Register

Risk	Likelihood	Impact	Mitigation
Scope creep across increments	High	Medium	Each increment has a frozen mini-SRS before coding begins.
Single-developer bottleneck	High	Medium	Strict incremental delivery keeps partial progress always demonstrable.
Render free-tier cold starts	Medium	Low	Documented limitation; pre-warm before live demos.
RLS misconfiguration exposing cross-role data	Medium	High	RLS policies unit-tested per role in Increment 1 (Section 3.7).
Supabase project pausing on inactivity	Medium	Low	Documented in deployment plan; periodic keep-alive ping.

0.8 Success Criteria

1. All four user roles can complete their primary workflow end-to-end on the deployed system.
2. Each increment has a corresponding SRS section, design note, test plan, and passing test cases.
3. The codebase demonstrably applies SOLID principles, verifiable in code review.
4. The system is deployed and publicly reachable on free-tier infrastructure.
5. This document traces the project continuously from feasibility through deployment and presentation.

PHASE 1 — SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1.1 Purpose

This SRS defines the complete functional and non-functional requirements for CampusEvents. It is written to IEEE-830-influenced structure, adapted for an incrementally delivered system: requirements are grouped both by category (functional/non-functional) and by the increment in which they will be delivered, so that each increment's later mini-spec (Sections 3–6) is traceable back to a requirement ID defined here.

1.2 Scope

CampusEvents is a web-based portal accessible to students, club organizers, faculty coordinators, and a super admin at a single college. It manages the full event lifecycle from creation through approval, publication, registration, attendance, and reporting. The system is composed of a Node.js/TypeScript backend API, a Supabase (Postgres) database with Row-Level Security, and a plain HTML/CSS/JS frontend that communicates with the backend exclusively via a REST API.

1.3 Definitions, Acronyms, Abbreviations

Term	Meaning
SRS	Software Requirements Specification
RLS	Row-Level Security (Postgres/Supabase access-control mechanism)
JWT	JSON Web Token, used by Supabase Auth for session/identity claims
Organizer	A user representing a club or committee who creates and manages events
Coordinator	A faculty member who approves or rejects events before publication
Increment	A shippable slice of the system delivered per the Incremental SDLC model
FR / NFR	Functional Requirement / Non-Functional Requirement (prefixed IDs used throughout)

1.4 User Roles

Role	Description
Student	Browses published events, registers, checks in via QR at the venue, receives certificates.
Organizer	Represents a club/committee. Creates and edits events, views registrants, manages check-in.
Coordinator	Faculty member who reviews submitted events and approves or rejects them with remarks.
Super Admin	Full system oversight: manages users/roles, views institution-wide analytics, moderates all events.

1.5 Functional Requirements

Each requirement is tagged with the increment that delivers it: [INC-1] Auth/Browsing/Registration/Creation, [INC-2] Approval/Notifications, [INC-3] Attendance/Certificates, [INC-4] Analytics/Feedback.

1.5.1 Authentication & Authorization

ID	Requirement	Increment
FR-1.1	The system shall allow a user to register with email, password, full name, and a role selection (Student or Organizer). Coordinator and Super Admin accounts are provisioned by a Super Admin, not self-registered.	INC-1
FR-1.2	The system shall allow a registered user to log in with email and password and receive a session (JWT).	INC-1
FR-1.3	The system shall restrict every API endpoint's accessible actions according to the requester's role, enforced at both the API layer and via Postgres RLS policies.	INC-1
FR-1.4	The system shall allow a user to log out, invalidating the client-held session.	INC-1

1.5.2 Event Browsing & Registration (Student)

ID	Requirement	Increment
FR-2.1	The system shall display a list of published, upcoming events to any authenticated user.	INC-1
FR-2.2	The system shall allow filtering the event list by category, date range, and organizing club.	INC-1
FR-2.3	The system shall allow a Student to view full event details (description, venue, timing, capacity, remaining seats).	INC-1
FR-2.4	The system shall allow a Student to register for an event, subject to capacity limits and a registration deadline.	INC-1
FR-2.5	The system shall prevent duplicate registration by the same student for the same event.	INC-1
FR-2.6	The system shall allow a Student to view their own list of registered/past events.	INC-1

1.5.3 Event Creation & Management (Organizer)

ID	Requirement	Increment
FR-3.1	The system shall allow an Organizer to create a draft event with title, description, category, venue, start/end time, capacity, and registration deadline.	INC-1
FR-3.2	The system shall allow an Organizer to edit or withdraw an event while it is in Draft or Pending Approval status.	INC-1

ID	Requirement	Increment
FR-3.3	The system shall allow an Organizer to view the list of students registered for their own events.	INC-1
FR-3.4	The system shall allow an Organizer to submit a Draft event for Coordinator approval, transitioning it to Pending status.	INC-2

1.5.4 Approval Workflow (Coordinator)

ID	Requirement	Increment
FR-4.1	The system shall present a Coordinator with a queue of Pending events awaiting review.	INC-2
FR-4.2	The system shall allow a Coordinator to approve a Pending event, transitioning it to Published.	INC-2
FR-4.3	The system shall allow a Coordinator to reject a Pending event with a mandatory remarks field, transitioning it to Rejected and visible to the Organizer.	INC-2
FR-4.4	The system shall record an immutable audit entry (actor, timestamp, action, remarks) for every approval/rejection.	INC-2
FR-4.5	The system shall notify the Organizer (in-app and email) when their event is approved or rejected.	INC-2
FR-4.6	The system shall notify registered Students (in-app and email) of material changes to an event they registered for (time/venue change, cancellation).	INC-2

1.5.5 Attendance & Certification

ID	Requirement	Increment
FR-5.1	The system shall generate a unique QR code per registration, retrievable by the registered Student.	INC-3
FR-5.2	The system shall allow an Organizer to scan/enter a QR code at the venue to mark a registrant as Attended.	INC-3
FR-5.3	The system shall prevent a QR code from being marked Attended more than once.	INC-3
FR-5.4	The system shall auto-generate a PDF participation certificate for each Student marked Attended, after the Organizer marks the event Completed.	INC-3
FR-5.5	The system shall allow a Student to download their certificates from their profile.	INC-3

1.5.6 Administration & Analytics

ID	Requirement	Increment
FR-6.1	The system shall allow a Super Admin to view, create, and deactivate any user account and reassign roles.	INC-4

ID	Requirement	Increment
FR-6.2	The system shall allow a Super Admin to view/moderate (approve, reject, unpublish) any event regardless of Coordinator action.	INC-4
FR-6.3	The system shall present a Super Admin dashboard showing: total events per period, registrations per club, average approval turnaround time, and attendance rate per event.	INC-4
FR-6.4	The system shall allow a Student to submit optional post-event feedback (rating 1–5 and free text) after an event is marked Completed.	INC-4
FR-6.5	The system shall display aggregated feedback ratings to the Organizer and Super Admin.	INC-4

1.6 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Security	All passwords are handled exclusively by Supabase Auth (never stored/hashed manually by the backend). All cross-role data access is enforced by Postgres RLS policies in addition to backend checks (defense-in-depth).
NFR-2	Performance	Event listing endpoints shall respond within 500ms under expected load (hundreds of concurrent users) on free-tier infrastructure, excluding Render cold-start latency.
NFR-3	Availability	The system targets best-effort availability appropriate to free-tier hosting; documented cold-start behavior is an accepted trade-off (Risk Register, Section 0.7).
NFR-4	Usability	Core workflows (browse, register, create, approve) shall be completable by a first-time user without external instructions, using conventional web UI patterns.
NFR-5	Maintainability	Backend code shall follow SOLID principles (Section 2.6) and be organized into clearly separated layers (routes, services, repositories) to support independent modification of each increment.
NFR-6	Portability	The frontend shall run in any evergreen browser without build tooling; the backend shall run on any Node.js 20+ runtime.
NFR-7	Data Integrity	Event capacity, registration deadlines, and duplicate-registration constraints shall be enforced at the database level (constraints/triggers), not solely in application code.
NFR-8	Auditability	All approval/rejection actions shall be permanently logged and never deleted, per FR-4.4.

1.7 Use Case Summary

Full use-case narratives are provided per increment in Sections 3–6 (each increment's mini-SRS). This table gives the overall map from actor to primary use cases, establishing traceability before the detailed design begins.

Actor	Primary Use Cases
Student	Register/Login • Browse & Filter Events • View Event Details • Register for Event • View My Events • Check In via QR • Download Certificate • Submit Feedback
Organizer	Register/Login • Create Event • Edit/Withdraw Event • Submit for Approval • View Registrants • Scan Attendance • Mark Event Completed • View Feedback Summary
Coordinator	Login • View Pending Queue • Approve Event • Reject Event with Remarks • View Audit Log
Super Admin	Login • Manage Users & Roles • Moderate Any Event • View Analytics Dashboard • View Institution-wide Audit Log

1.8 Role–Permission Matrix

This matrix is the authoritative source for both the backend authorization checks and the Postgres RLS policies designed in Section 2.4. Any discrepancy between code and this table during review is treated as a defect.

Action	Student	Organizer	Coordinator	Super Admin
View published events	Yes	Yes	Yes	Yes
Register for event	Yes	No	No	No
Create/edit own draft event	No	Yes	No	Yes
Submit event for approval	No	Yes	No	Yes
Approve/reject event	No	No	Yes	Yes
View own registrants	No	Yes (own events)	No	Yes (all)
Mark attendance	No	Yes (own events)	No	Yes (all)
Manage users/roles	No	No	No	Yes
View analytics dashboard	No	No	No	Yes

1.9 Assumptions & Constraints

- Single-institution deployment; no multi-tenancy.
- Users have valid college email addresses; email verification is delegated to Supabase Auth's built-in flow.
- Free-tier hosting limits (Section 0.3.2) are accepted constraints, not defects.
- Payment handling, native mobile apps, and real-time chat are out of scope per Section 0.5.

PHASE 2 — SYSTEM DESIGN

2.1 Architectural Overview

CampusEvents follows a strict three-tier separation, chosen specifically so the HTML/CSS/JS frontend never touches Supabase or business logic directly — it is a thin client that only calls the backend.

- **Presentation Tier:** static HTML/CSS/vanilla JS. No frontend framework. JS modules call backend REST endpoints via `fetch()` and render responses. No Supabase client, no secrets, no business logic here.
- **Application Tier:** Node.js + TypeScript backend (Express-style REST API). Owns all business logic, input validation, and orchestration. Holds the Supabase service-role key (server-side only) for privileged operations, and forwards the user's JWT for RLS-scoped operations.
- **Data Tier:** Supabase (Postgres). Row-Level Security policies are the primary authorization boundary (Section 2.4); the schema also enforces structural invariants (capacity, no duplicate registration) via constraints and triggers, so integrity does not depend solely on application code.

This separation directly implements NFR-1 (defense-in-depth security) and NFR-5 (maintainability): each increment adds new routes/services/policies without the frontend needing anything beyond a new `fetch()` call.

2.2 High-Level Component Diagram (described)

Browser (HTML/CSS/JS) → HTTPS/JSON → Node.js/TypeScript API (Auth middleware → Route → Service → Repository) → Supabase client (service-role for admin ops, user-JWT-scoped for RLS-governed ops) → Postgres (tables + RLS policies + triggers). Outbound: Email notifications via a provider adapter (e.g. Resend/SMTP) called from the Notification Service; PDF certificates generated server-side and stored in Supabase Storage.

2.3 Entity-Relationship Design

Six core tables model the full event lifecycle: profiles (extends Supabase `auth.users` with app role), events, event_approvals (audit trail, FR-4.4), registrations (with a unique QR code and a DB-level uniqueness constraint enforcing FR-2.5), certificates, and feedback.

Table	Key Relationships
profiles	1:1 with <code>auth.users</code> ; 1:many to events (as organizer) and registrations (as student)
events	many:1 to profiles (<code>organizer_id</code>); 1:many to registrations and event_approvals
event_approvals	many:1 to events and profiles (<code>coordinator_id</code>) — append-only audit log
registrations	many:1 to events and profiles (<code>student_id</code>); 1:1 to certificates and feedback
certificates	1:1 to registrations, created only after event completion + attendance
feedback	1:1 to registrations, optional, only after event completion

Schema Definition (PostgreSQL / Supabase)

```
-- Core entity: users mirrored from Supabase Auth, extended with app-specific role
create type user_role as enum ('student', 'organizer', 'coordinator', 'super_admin');
```

```

create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null,
  role_user_role not null default 'student',
  club_name text,           -- populated for organizers
  created_at timestampz not null default now()
);

create type event_status as enum (
  'draft', 'pending', 'published', 'rejected', 'completed', 'cancelled'
);

create table events (
  id uuid primary key default gen_random_uuid(),
  organizer_id uuid not null references profiles(id),
  title text not null,
  description text not null,
  category text not null,
  venue text not null,
  start_time timestampz not null,
  end_time timestampz not null,
  registration_deadline timestampz not null,
  capacity integer not null check (capacity > 0),
  status event_status not null default 'draft',
  created_at timestampz not null default now(),
  constraint valid_window check (end_time > start_time),
  constraint valid_deadline check (registration_deadline <= start_time)
);

create table event_approvals (
  id uuid primary key default gen_random_uuid(),
  event_id uuid not null references events(id) on delete cascade,
  coordinator_id uuid not null references profiles(id),
  decision text not null check (decision in ('approved', 'rejected')),
  remarks text,
  decided_at timestampz not null default now()
);

create table registrations (
  id uuid primary key default gen_random_uuid(),
  event_id uuid not null references events(id) on delete cascade,
  student_id uuid not null references profiles(id),
  qr_code text not null unique,
  attended boolean not null default false,
  attended_at timestampz,
  registered_at timestampz not null default now(),
  unique (event_id, student_id)      -- enforces FR-2.5, no duplicate registration
);

create table certificates (
  id uuid primary key default gen_random_uuid(),
  registration_id uuid not null references registrations(id) on delete cascade unique,
  issued_at timestampz not null default now(),
  pdf_url text not null
);

create table feedback (
  id uuid primary key default gen_random_uuid(),
  registration_id uuid not null references registrations(id) on delete cascade unique,
  rating integer not null check (rating between 1 and 5),
  comment text,

```

```

    submitted_at timestamptz not null default now()
);

-- Capacity enforcement at the DB level (NFR-7): trigger blocking over-capacity inserts
create or replace function check_event_capacity() returns trigger as $$
declare
    current_count integer;
    max_capacity integer;
begin
    select count(*), e.capacity into current_count, max_capacity
    from registrations r join events e on e.id = r.event_id
    where r.event_id = new.event_id
    group by e.capacity;

    if current_count >= max_capacity then
        raise exception 'Event capacity reached';
    end if;
    return new;
end;
$$ language plpgsql;

create trigger enforce_capacity
    before insert on registrations
    for each row execute function check_event_capacity();

```

schema.sql — core tables, enums, constraints, and the capacity-enforcement trigger (implements NFR-7).

2.4 Authorization Design: RLS-First

Per the architecture decision in Section 0.4, Postgres Row-Level Security is the primary access-control mechanism, not the backend alone. This means even if a backend authorization check were ever missed or buggy, the database itself refuses unauthorized reads/writes — directly satisfying NFR-1. The backend service layer still performs the same checks (Section 2.6, Open/Closed and Dependency Inversion discussion) to fail fast with clean error messages, but RLS is the actual security boundary.

```

-- RLS-first authorization (Section 0.4 decision): Postgres enforces access
-- as the primary defense; the backend service layer duplicates checks for
-- fast-fail UX, but RLS is what actually protects the data.

alter table events enable row level security;
alter table registrations enable row level security;
alter table event_approvals enable row level security;

-- Anyone authenticated can read PUBLISHED events
create policy events_select_published on events
    for select using (status = 'published');

-- Organizers can read/write their own events regardless of status
create policy events_owner_all on events
    for all using (
        organizer_id = auth.uid()
        or exists (select 1 from profiles p where p.id = auth.uid() and p.role = 'super_admin')
    );

-- Coordinators can read PENDING events (for the approval queue)
create policy events_select_pending_coordinator on events
    for select using (
        status = 'pending'
        and exists (select 1 from profiles p where p.id = auth.uid() and p.role in ('coordinator',
'super_admin'))
    );

```

```
-- Students can insert their own registration only
create policy registrations_insert_self on registrations
  for insert with check (student_id = auth.uid());

-- Students can view only their own registrations; organizers can view
-- registrations for events they own; super_admin sees all
create policy registrations_select_scoped on registrations
  for select using (
    student_id = auth.uid()
  or exists (
    select 1 from events e
    where e.id = registrations.event_id and e.organizer_id = auth.uid()
  )
  or exists (select 1 from profiles p where p.id = auth.uid() and p.role = 'super_admin')
);
```

rls_policies.sql — representative policies; the full set is completed per-increment as each table becomes active (Sections 3–6).

2.5 Data Flow Diagrams (described)

DFD Level 0 (Context Diagram)

A single process, 'CampusEvents System', with four external entities — Student, Organizer, Coordinator, Super Admin — each exchanging requests/responses with the system. Students send registration requests and receive event data/certificates; Organizers send event data and receive registrant lists; Coordinators send approval decisions and receive the pending queue; Super Admin sends moderation/user-management commands and receives analytics data.

DFD Level 1 (Major Processes)

- P1 — Manage Authentication: validates credentials against Supabase Auth, issues sessions.
- P2 — Manage Events: handles create/edit/submit, reads/writes the events table.
- P3 — Manage Approvals: reads pending events, writes decisions to event_approvals, triggers P5.
- P4 — Manage Registrations: validates capacity/deadline, writes registrations, generates QR.
- P5 — Notify: reads state transitions from P3/P4, sends email/in-app notifications.
- P6 — Manage Attendance & Certificates: validates QR, writes attendance, generates PDF certificates.
- P7 — Analytics: aggregates read-only queries across events/registrations/feedback for the admin dashboard.

2.6 API Contract

All endpoints are prefixed `/api` and require a valid Supabase JWT in the Authorization header except registration and login. Each endpoint's role restriction (column 2) is enforced identically in backend middleware and in the corresponding RLS policy.

Method & Path	Roles	Purpose	Increment
POST <code>/api/auth/register</code>	Public	Create account (student/organizer)	INC-1
POST <code>/api/auth/login</code>	Public	Authenticate, return session	INC-1
GET <code>/api/events</code>	All authenticated	List published events (filterable)	INC-1

Method & Path	Roles	Purpose	Increment
GET /api/events/:id	All authenticated	Event detail	INC-1
POST /api/events	Organizer, SuperAdmin	Create draft event	INC-1
PATCH /api/events/:id	Owner Organizer, SuperAdmin	Edit/withdraw draft event	INC-1
POST /api/events/:id/register	Student	Register for an event	INC-1
GET /api/me/registrations	Student	My registered events	INC-1
POST /api/events/:id/submit	Owner Organizer	Submit draft for approval	INC-2
GET /api/approvals/pending	Coordinator, SuperAdmin	Approval queue	INC-2
POST /api/approvals/:eventId/decide	Coordinator, SuperAdmin	Approve/reject with remarks	INC-2
POST /api/registrations/:id/checkin	Owner Organizer	Mark attendance via QR	INC-3
POST /api/events/:id/complete	Owner Organizer	Mark event Completed, trigger certs	INC-3
GET /api/me/certificates	Student	List/download certificates	INC-3
GET /api/admin/analytics	SuperAdmin	Dashboard metrics	INC-4
POST /api/registrations/:id/feedback	Student	Submit feedback	INC-4

2.7 Frontend Structure (HTML/CSS/JS)

The frontend is organized as a small set of static pages, each backed by a single JS module that only calls the backend API — never Supabase directly, per the architecture rule in Section 2.1.

Page	JS Module	Calls
index.html (event list)	events.js	GET /api/events, GET /api/events/:id
login.html / register.html	auth.js	POST /api/auth/login, POST /api/auth/register
organizer-dashboard.html	organizer.js	POST/PATCH /api/events, GET registrants, check-in
coordinator-queue.html	coordinator.js	GET /api/approvals/pending, POST decide
admin-dashboard.html	admin.js	GET /api/admin/analytics, user management endpoints
my-events.html	profile.js	GET /api/me/registrations, /api/me/certificates

2.8 Wireframes (described)

Low-fidelity layout intent for the two highest-traffic screens, sufficient for implementation without a dedicated design tool:

Event List (Student view)

- Top bar: logo, nav links (Browse, My Events, Profile), logout.
- Filter row: category dropdown, date-range picker, club search box.
- Card grid: each card shows title, club, date/venue, seats remaining, 'View Details' button.

Organizer Dashboard

- Left sidebar: My Events (Draft/Pending/Published/Completed tabs).
- Main panel: selected event's form (create/edit) or registrant table with check-in action.
- Prominent 'Submit for Approval' button, disabled until required fields are valid.

2.9 SOLID Principles — Governing Design Rules for All Code

Every increment's backend code (Sections 3.6, 4.5, 5.5, 6.5) is written and reviewed against these five principles. They are stated once here as the standing rule set referenced throughout the rest of this document.

S — Single Responsibility Principle

Each class/module has exactly one reason to change. Routes only parse HTTP and delegate; Services contain business rules; Repositories only perform data access. An EventService never constructs an HTTP response, and a route handler never writes SQL/Supabase calls directly.

O — Open/Closed Principle

Modules are open for extension, closed for modification. Notification channels (email, in-app) are implemented as interchangeable strategies behind a common NotificationChannel interface, so Increment 2 can add SMS later without editing existing channel code.

L — Liskov Substitution Principle

Any implementation of an interface (e.g. a NotificationChannel or a Repository) must be substitutable without breaking callers. Tests for services are written against the interface, and any concrete implementation (real Supabase repository, or an in-memory fake for unit tests) must pass the same behavioral contract.

I — Interface Segregation Principle

Interfaces are kept narrow and role-specific rather than one large repository interface. An EventReader and EventWriter are separated where a consumer only needs one, so implementers are not forced to satisfy methods they don't use.

D — Dependency Inversion Principle

Services depend on abstractions (repository/interfaces), not concrete Supabase client calls, injected at construction time. This is what allows RLS-scoped vs service-role Supabase clients (Section 2.4) to be swapped per-request without touching service logic, and allows unit tests to run against fakes instead of a live database.

2.10 Deployment Architecture (preview)

Full deployment steps are detailed in Section 7 (Deployment Plan). In summary: the frontend static files are hosted on Vercel; the Node.js/TypeScript backend is hosted on Render as a web service; Supabase hosts Postgres, Auth, and Storage (for certificate PDFs and QR assets). Environment variables (Supabase URL, anon key, service-role key, JWT secret) are configured per-environment and never committed to source control.

PHASE 3 — INCREMENT 1: AUTH, BROWSING, REGISTRATION, CREATION

3.1 Increment Goal

Deliver a fully working, independently deployable slice: users can register/log in with a role, students can browse and register for published events, and organizers can create and edit draft events. This increment corresponds to the SRS requirements tagged [INC-1] (Sections 1.5.1–1.5.3) and satisfies the Incremental model's requirement that each increment be 'a meaningful addition to the system that can be integrated, tested, and delivered' rather than unfinished code.

3.2 Mini-Requirements (frozen scope for this increment)

Per the Risk Register (Section 0.7), scope for this increment is frozen to exactly the requirements below before coding begins.

ID	Requirement (abridged)
FR-1.1 – FR-1.4	Register, login, role-based authorization, logout
FR-2.1 – FR-2.6	Browse/filter/view published events; register; view my registrations; no duplicates
FR-3.1 – FR-3.3	Organizer creates/edits draft events; views own registrants

3.3 Increment Design Notes

This increment introduces the domain types, error hierarchy, repository interfaces, and their Supabase-backed implementations that all later increments build on — this is deliberate: getting the abstractions right here (Section 2.9's SOLID rules) means Increments 2–4 add new services and routes without modifying this layer, only extending it.

- Domain layer: plain TypeScript types/interfaces, no dependencies (Section 3.6.1–3.6.2).
- Repository layer: narrow, role-appropriate interfaces (ISP) with a Supabase-backed implementation (Section 3.6.3–3.6.4).
- Service layer: `EventService` and `RegistrationService` hold all business rules, depend only on repository interfaces (DIP) (Section 3.6.5–3.6.6).
- Route layer: thin HTTP adapters with centralized error mapping (Section 3.6.7).
- Frontend: a single `events.js` module demonstrating the 'frontend calls backend only' rule (Section 3.6.9).

3.4 Sequence: Student Registers for an Event

1. Browser calls `GET /api/events/:id` to show details (frontend: `events.js`).
2. Browser calls `POST /api/events/:id/register` with the session JWT.
3. `requireAuth` middleware validates the JWT and attaches `req.user`.
4. Route delegates to `RegistrationService.register(actor, eventId)`.
5. Service checks role, event status, deadline, duplicate, and capacity — in that order — throwing a typed `DomainError` on the first failure.
6. On success, `RegistrationRepository.create()` inserts the row; the DB's `unique(event_id, student_id)` constraint and capacity trigger (Section 2.3) back up the application-level checks.

7. Response returns the registration including its QR code (used later in Increment 3).

3.5 Frozen Test Plan for This Increment

Test ID	Scenario	Expected Result	Traces to
T1.1	Organizer creates a valid draft event	201, status = draft	FR-3.1
T1.2	Student attempts to create an event	403 Forbidden	Role-Permission Matrix
T1.3	Event created with end time before start time	400 Validation error	NFR-7 (data integrity)
T1.4	Student registers for a published event with free seats	201, registration returned with QR	FR-2.4
T1.5	Student registers twice for the same event	409 Duplicate registration	FR-2.5
T1.6	Student registers once capacity is full	409 Capacity exceeded	NFR-7
T1.7	Student registers after the registration deadline	400 Validation error	FR-2.4
T1.8	Unauthenticated request to any protected route	401 Unauthorized	NFR-1

3.6 Implementation

3.6.1 Domain Types

```
// domain/types.ts
// SRP: this module's only responsibility is defining shared domain shapes.
// No business logic, no I/O – pure types, so every other module can depend
// on it without pulling in unrelated behavior (supports ISP downstream).

export type UserRole = "student" | "organizer" | "coordinator" | "super_admin";

export interface Profile {
  id: string;
  fullName: string;
  role: UserRole;
  clubName?: string;
}

export type EventStatus =
  | "draft" | "pending" | "published" | "rejected" | "completed" | "cancelled";

export interface EventRecord {
  id: string;
  organizerId: string;
  title: string;
  description: string;
  category: string;
  venue: string;
  startTime: string; // ISO 8601
  endTime: string;
  registrationDeadline: string;
}
```

```

    capacity: number;
    status: EventStatus;
    createdAt: string;
}

export interface CreateEventInput {
    title: string;
    description: string;
    category: string;
    venue: string;
    startTime: string;
    endTime: string;
    registrationDeadline: string;
    capacity: number;
}

export interface Registration {
    id: string;
    eventId: string;
    studentId: string;
    qrCode: string;
    attended: boolean;
    registeredAt: string;
}

```

domain/types.ts

3.6.2 Domain Errors

```

// domain/errors.ts
// SRP: centralizes the small set of domain error types so services throw
// meaningful, typed errors instead of generic Error, and route handlers
// (Section 3.6.4) can map them to correct HTTP status codes (SRS NFR-1/FR).

export class DomainError extends Error {
    constructor(message: string, public readonly code: string) {
        super(message);
        this.name = "DomainError";
    }
}

export class NotFoundError extends DomainError {
    constructor(entity: string) {
        super(`${entity} not found`, "NOT_FOUND");
    }
}

export class ForbiddenError extends DomainError {
    constructor(message = "You are not allowed to perform this action") {
        super(message, "FORBIDDEN");
    }
}

export class ValidationError extends DomainError {
    constructor(message: string) {
        super(message, "VALIDATION_ERROR");
    }
}

export class CapacityExceededError extends DomainError {
    constructor() {
        super("Event has reached capacity", "CAPACITY_EXCEEDED");
    }
}

```

```

}

export class DuplicateRegistrationError extends DomainError {
  constructor() {
    super("You have already registered for this event", "DUPLICATE_REGISTRATION");
  }
}

```

domain/errors.ts

3.6.3 Repository Interfaces (Interface Segregation)

```

// repositories/interfaces.ts
// ISP: two narrow interfaces instead of one bloated "EventRepository" with
// every possible method. EventReader is all the browsing/registration flow
// needs; EventWriter is only needed by the create/edit flow. A future
// read-only analytics module (Increment 4) can depend on EventReader alone.
// DIP: services will depend on these abstractions, never on the concrete
// Supabase client directly.

import { EventRecord, CreateEventInput, Registration } from "../domain/types";

export interface EventReader {
  findPublished(filters: EventFilters): Promise<EventRecord[]>;
  findById(id: string): Promise<EventRecord | null>;
}

export interface EventWriter {
  create(organizerId: string, input: CreateEventInput): Promise<EventRecord>;
  update(id: string, patch: Partial<CreateEventInput>): Promise<EventRecord>;
}

export interface EventFilters {
  category?: string;
  club?: string;
  fromDate?: string;
  toDate?: string;
}

export interface RegistrationRepository {
  create(eventId: string, studentId: string, qrCode: string): Promise<Registration>;
  countForEvent(eventId: string): Promise<number>;
  existsForStudent(eventId: string, studentId: string): Promise<boolean>;
  findByStudent(studentId: string): Promise<Registration[]>;
}

```

repositories/interfaces.ts

3.6.4 Supabase Repository Implementation

```

// repositories/supabaseEventRepository.ts
// SRP: this class's only job is translating EventReader/EventWriter calls
// into Supabase queries. No validation, no authorization decisions here –
// those live in the service layer (EventService below). This is the
// concrete implementation that gets injected wherever EventReader/
// EventWriter is required (DIP in action).

import { SupabaseClient } from "@supabase/supabase-js";
import { EventReader, EventWriter, EventFilters } from "../interfaces";
import { EventRecord, CreateEventInput } from "../domain/types";
import { NotFoundError } from "../domain/errors";

export class SupabaseEventRepository implements EventReader, EventWriter {

```

```

// The Supabase client is injected, not constructed here – this is what
// lets Increment 1's tests substitute a fake client (LSP: any client
// satisfying the same query surface is substitutable).
constructor(private readonly db: SupabaseClient) {}

async findPublished(filters: EventFilters): Promise<EventRecord[]> {
  let query = this.db
    .from("events")
    .select("*")
    .eq("status", "published")
    .order("start_time", { ascending: true });

  if (filters.category) query = query.eq("category", filters.category);
  if (filters.club) query = query.eq("club_name", filters.club);
  if (filters.fromDate) query = query.gte("start_time", filters.fromDate);
  if (filters.toDate) query = query.lte("start_time", filters.toDate);

  const { data, error } = await query;
  if (error) throw error;
  return (data ?? []).map(mapRow);
}

async findById(id: string): Promise<EventRecord | null> {
  const { data, error } = await this.db
    .from("events").select("*").eq("id", id).maybeSingle();
  if (error) throw error;
  return data ? mapRow(data) : null;
}

async create(organizerId: string, input: CreateEventInput): Promise<EventRecord> {
  const { data, error } = await this.db
    .from("events")
    .insert({ organizer_id: organizerId, ...toRow(input), status: "draft" })
    .select("*").single();
  if (error) throw error;
  return mapRow(data);
}

async update(id: string, patch: Partial<CreateEventInput>): Promise<EventRecord> {
  const { data, error } = await this.db
    .from("events").update(toRow(patch)).eq("id", id).select("*").maybeSingle();
  if (error) throw error;
  if (!data) throw new NotFoundError("Event");
  return mapRow(data);
}
}

// Pure mapping helpers, kept outside the class body since they hold no
// state and don't belong to the repository's single responsibility list.
function mapRow(row: any): EventRecord {
  return {
    id: row.id, organizerId: row.organizer_id, title: row.title,
    description: row.description, category: row.category, venue: row.venue,
    startTime: row.start_time, endTime: row.end_time,
    registrationDeadline: row.registration_deadline, capacity: row.capacity,
    status: row.status, createdAt: row.created_at,
  };
}

function toRow(input: Partial<CreateEventInput>): Record<string, unknown> {
  const row: Record<string, unknown> = {};
  if (input.title !== undefined) row.title = input.title;
  if (input.description !== undefined) row.description = input.description;

```

```

    if (input.category !== undefined) row.category = input.category;
    if (input.venue !== undefined) row.venue = input.venue;
    if (input.startTime !== undefined) row.start_time = input.startTime;
    if (input.endTime !== undefined) row.end_time = input.endTime;
    if (input.registrationDeadline !== undefined) row.registration_deadline =
input.registrationDeadline;
    if (input.capacity !== undefined) row.capacity = input.capacity;
    return row;
}

```

repositories/supabaseEventRepository.ts

3.6.5 Event Service (*business rules*)

```

// services/eventService.ts
// SRP: owns event business rules (validation, authorization decisions,
// lifecycle transitions) and nothing else – it never touches SQL or HTTP.
// DIP: depends on the EventReader/EventWriter abstractions, injected via
// the constructor, not on SupabaseEventRepository directly. This means
// EventService is fully unit-testable with an in-memory fake (Section 3.7).

import { EventReader, EventWriter, EventFilters } from "../repositories/interfaces";
import { EventRecord, CreateEventInput, Profile } from "../domain/types";
import { ForbiddenError, NotFoundError, ValidationError } from "../domain/errors";

export class EventService {
  constructor(
    private readonly reader: EventReader,
    private readonly writer: EventWriter
  ) {}

  async listPublished(filters: EventFilters): Promise<EventRecord[]> {
    return this.reader.findPublished(filters);
  }

  async getById(id: string): Promise<EventRecord> {
    const event = await this.reader.findById(id);
    if (!event) throw new NotFoundError("Event");
    return event;
  }

  async createDraft(actor: Profile, input: CreateEventInput): Promise<EventRecord> {
    // Authorization: only Organizer or SuperAdmin may create events (FR-3.1,
    // Role-Permission Matrix Section 1.8). Mirrors the RLS policy in
    // Section 2.4 – defense-in-depth, not the sole check.
    if (actor.role !== "organizer" && actor.role !== "super_admin") {
      throw new ForbiddenError("Only organizers may create events");
    }
    this.validateEventWindow(input);
    return this.writer.create(actor.id, input);
  }

  async updateDraft(actor: Profile, eventId: string, patch: Partial<CreateEventInput>):
Promise<EventRecord> {
    const existing = await this.getById(eventId);
    if (existing.organizerId !== actor.id && actor.role !== "super_admin") {
      throw new ForbiddenError("You do not own this event");
    }
    if (existing.status !== "draft" && existing.status !== "pending") {
      throw new ValidationError("Only draft or pending events can be edited");
    }
    return this.writer.update(eventId, patch);
  }
}

```



```

// Extracted so validation logic has one owner and one location – any
// future rule change (e.g. minimum lead time) touches only this method,
// satisfying Open/Closed at the method level within a stable class shape.
private validateEventWindow(input: CreateEventInput): void {
  const start = new Date(input.startTime).getTime();
  const end = new Date(input.endTime).getTime();
  const deadline = new Date(input.registrationDeadline).getTime();

  if (Number.isNaN(start) || Number.isNaN(end) || Number.isNaN(deadline)) {
    throw new ValidationError("Invalid date(s) supplied");
  }
  if (end <= start) throw new ValidationError("End time must be after start time");
  if (deadline > start) throw new ValidationError("Registration deadline must be before start
time");
  if (input.capacity <= 0) throw new ValidationError("Capacity must be positive");
}
}

```

services/eventService.ts

3.6.6 Registration Service (business rules)

```

// services/registrationService.ts
// SRP: owns registration business rules only. Delegates event lookups to
// EventReader and QR generation to a QrCodeGenerator abstraction (OCP: a
// new QR provider can be swapped in without touching this class).

import { EventReader } from "../repositories/interfaces";
import { RegistrationRepository } from "../repositories/interfaces";
import { Registration, Profile } from "../domain/types";
import {
  CapacityExceededError, DuplicateRegistrationError,
  NotFoundError, ValidationError, ForbiddenError,
} from "../domain/errors";

export interface QrCodeGenerator {
  generate(payload: string): string; // returns an opaque unique code/token
}

export class RegistrationService {
  constructor(
    private readonly events: EventReader,
    private readonly registrations: RegistrationRepository,
    private readonly qr: QrCodeGenerator
  ) {}

  async register(actor: Profile, eventId: string): Promise<Registration> {
    if (actor.role !== "student") {
      throw new ForbiddenError("Only students may register for events");
    }
    const event = await this.events.findById(eventId);
    if (!event) throw new NotFoundError("Event");
    if (event.status !== "published") {
      throw new ValidationError("Event is not open for registration");
    }
    if (new Date(event.registrationDeadline).getTime() < Date.now()) {
      throw new ValidationError("Registration deadline has passed");
    }

    const alreadyRegistered = await this.registrations.existsForStudent(eventId, actor.id);
    if (alreadyRegistered) throw new DuplicateRegistrationError(); // enforces FR-2.5
  }
}

```

```

    const currentCount = await this.registrations.countForEvent(eventId);
    if (currentCount >= event.capacity) throw new CapacityExceededError();

    const qrCode = this.qr.generate(`${eventId}:${actor.id}:${Date.now()}`);
    return this.registrations.create(eventId, actor.id, qrCode);
  }

  async myRegistrations(actor: Profile): Promise<Registration[]> {
    return this.registrations.findByStudent(actor.id);
  }
}

```

services/registrationService.ts

3.6.7 Routes (HTTP adapter layer)

```

// routes/events.ts
// SRP: this module's only responsibility is HTTP plumbing – parse the
// request, call the service, shape the response. No business rules live
// here; every decision is delegated to EventService/RegistrationService.
// Errors are mapped centrally so every route behaves consistently (NFR-1).

import { Router, Request, Response, NextFunction } from "express";
import { EventService } from "../services/eventService";
import { RegistrationService } from "../services/registrationService";
import { requireAuth, requireRole } from "../middleware/auth";
import { DomainError } from "../domain/errors";

export function buildEventRoutes(
  eventService: EventService,
  registrationService: RegistrationService
): Router {
  const router = Router();

  router.get("/events", requireAuth, async (req: Request, res: Response, next: NextFunction) => {
    try {
      const { category, club, fromDate, toDate } = req.query as Record<string, string>;
      const events = await eventService.listPublished({ category, club, fromDate, toDate });
      res.json({ data: events });
    } catch (err) { next(err); }
  });

  router.get("/events/:id", requireAuth, async (req, res, next) => {
    try {
      const event = await eventService.getById(req.params.id);
      res.json({ data: event });
    } catch (err) { next(err); }
  });

  router.post("/events", requireAuth, requireRole("organizer", "super_admin"), async (req, res,
next) => {
    try {
      const event = await eventService.createDraft(req.user!, req.body);
      res.status(201).json({ data: event });
    } catch (err) { next(err); }
  });

  router.patch("/events/:id", requireAuth, requireRole("organizer", "super_admin"), async (req,
res, next) => {
    try {
      const event = await eventService.updateDraft(req.user!, req.params.id, req.body);
      res.json({ data: event });
    } catch (err) { next(err); }
  });
}

```

```

    });

    router.post("/events/:id/register", requireAuth, requireRole("student"), async (req, res, next)
=> {
    try {
        const registration = await registrationService.register(req.user!, req.params.id);
        res.status(201).json({ data: registration });
    } catch (err) { next(err); }
    });

    router.get("/me/registrations", requireAuth, requireRole("student"), async (req, res, next) => {
    try {
        const registrations = await registrationService.myRegistrations(req.user!);
        res.json({ data: registrations });
    } catch (err) { next(err); }
    });

    return router;
}

// Central error mapper (mounted once at app level, referenced here for
// context): converts DomainError subtypes to correct HTTP status codes.
export function errorHandler(err: unknown, _req: Request, res: Response, _next: NextFunction) {
    if (err instanceof DomainError) {
        const statusByCode: Record<string, number> = {
            NOT_FOUND: 404, FORBIDDEN: 403, VALIDATION_ERROR: 400,
            CAPACITY_EXCEEDED: 409, DUPLICATE_REGISTRATION: 409,
        };
        return res.status(statusByCode[err.code] ?? 400).json({ error: err.message, code: err.code });
    }
    console.error(err); // logging: unexpected errors are logged server-side, not leaked to client
    return res.status(500).json({ error: "Internal server error" });
}

```

routes/events.ts

3.6.8 Unit Tests

```

// tests/eventService.test.ts
// LSP in practice: FakeEventRepository below implements exactly the same
// EventReader/EventWriter interfaces the real SupabaseEventRepository does,
// so EventService is tested without any network/database dependency, and
// the test proves the service is correct regardless of which
// implementation is substituted in.

import { describe, it, expect, beforeEach } from "vitest";
import { EventService } from "../services/eventService";
import { EventReader, EventWriter, EventFilters } from "../repositories/interfaces";
import { EventRecord, CreateEventInput, Profile } from "../domain/types";
import { ForbiddenError, ValidationError } from "../domain/errors";

class FakeEventRepository implements EventReader, EventWriter {
    events: EventRecord[] = [];
    async findPublished(_f: EventFilters) { return this.events.filter(e => e.status ===
"published"); }
    async findById(id: string) { return this.events.find(e => e.id === id) ?? null; }
    async create(organizerId: string, input: CreateEventInput): Promise<EventRecord> {
        const event: EventRecord = { id: crypto.randomUUID(), organizerId, status: "draft", createdAt:
new Date().toISOString(), ...input };
        this.events.push(event);
        return event;
    }
    async update(id: string, patch: Partial<CreateEventInput>): Promise<EventRecord> {

```

```

        const event = this.events.find(e => e.id === id!);
        Object.assign(event, patch);
        return event;
    }
}

const organizer: Profile = { id: "org-1", fullName: "Coding Club", role: "organizer" };
const student: Profile = { id: "stu-1", fullName: "A Student", role: "student" };

const validInput: CreateEventInput = {
    title: "Hackathon", description: "24hr build", category: "Technical",
    venue: "Main Hall", startTime: "2026-10-10T09:00:00Z", endTime: "2026-10-11T09:00:00Z",
    registrationDeadline: "2026-10-08T00:00:00Z", capacity: 100,
};

describe("EventService.createDraft", () => {
    let repo: FakeEventRepository;
    let service: EventService;
    beforeEach(() => { repo = new FakeEventRepository(); service = new EventService(repo, repo); });

    it("allows an organizer to create a draft event", async () => {
        const event = await service.createDraft(organizer, validInput);
        expect(event.status).toBe("draft");
        expect(event.organizerId).toBe("org-1");
    });

    it("rejects creation by a student (FR-3.1 / Role-Permission Matrix)", async () => {
        await expect(service.createDraft(student, validInput)).rejects.toBeInstanceOf(ForbiddenError);
    });

    it("rejects an event whose end time precedes its start time", async () => {
        const bad = { ...validInput, endTime: "2026-10-09T00:00:00Z" };
        await expect(service.createDraft(organizer, bad)).rejects.toBeInstanceOf(ValidationError);
    });

    it("rejects a registration deadline after the event start", async () => {
        const bad = { ...validInput, registrationDeadline: "2026-10-10T10:00:00Z" };
        await expect(service.createDraft(organizer, bad)).rejects.toBeInstanceOf(ValidationError);
    });
});

```

tests/eventService.test.ts

```

// tests/registrationService.test.ts
// Covers FR-2.4 (register), FR-2.5 (no duplicate registration), and the
// DB-level capacity trigger's application-layer counterpart.

import { describe, it, expect, beforeEach } from "vitest";
import { RegistrationService } from "../services/registrationService";
import { EventReader } from "../repositories/interfaces";
import { EventRecord, Profile } from "../domain/types";
import { CapacityExceededError, DuplicateRegistrationError, ValidationError } from
"../domain/errors";

class FakeEventReader implements Pick<EventReader, "findById" | "findPublished"> {
    constructor(private event: EventRecord) {}
    async findById(id: string) { return id === this.event.id ? this.event : null; }
    async findPublished() { return [this.event]; }
}

class FakeRegistrationRepo {
    rows: { eventId: string; studentId: string }[] = [];
}

```

```

    async existsForStudent(eventId: string, studentId: string) {
      return this.rows.some(r => r.eventId === eventId && r.studentId === studentId);
    }
    async countForEvent(eventId: string) { return this.rows.filter(r => r.eventId === eventId).length; }
    async create(eventId: string, studentId: string, qrCode: string) {
      this.rows.push({ eventId, studentId });
      return { id: crypto.randomUUID(), eventId, studentId, qrCode, attended: false, registeredAt: new Date().toISOString() };
    }
    async findByStudent(studentId: string) { return this.rows.filter(r => r.studentId === studentId) as any; }
  }

const student: Profile = { id: "stu-1", fullName: "A Student", role: "student" };
const publishedEvent: EventRecord = {
  id: "evt-1", organizerId: "org-1", title: "Hackathon", description: "", category: "Tech",
  venue: "Hall", startTime: "2027-01-01T09:00:00Z", endTime: "2027-01-01T18:00:00Z",
  registrationDeadline: "2026-12-30T00:00:00Z", capacity: 1, status: "published", createdAt: "",
};

describe("RegistrationService.register", () => {
  let repo: FakeRegistrationRepo;
  let service: RegistrationService;
  beforeEach(() => {
    repo = new FakeRegistrationRepo();
    service = new RegistrationService(
      new FakeEventReader(publishedEvent) as any, repo as any, { generate: (p: string) => `qr-${p}` }
    );
  });

  it("registers a student for a published event within capacity", async () => {
    const reg = await service.register(student, "evt-1");
    expect(reg.studentId).toBe("stu-1");
  });

  it("rejects a duplicate registration by the same student", async () => {
    await service.register(student, "evt-1");
    await expect(service.register(student, "evt-1")).rejects.toBeInstanceOf(DuplicateRegistrationError);
  });

  it("rejects registration once capacity is reached", async () => {
    await service.register(student, "evt-1");
    const other: Profile = { id: "stu-2", fullName: "B Student", role: "student" };
    await expect(service.register(other, "evt-1")).rejects.toBeInstanceOf(CapacityExceededError);
  });

  it("rejects registration after the deadline has passed", async () => {
    const expired = { ...publishedEvent, registrationDeadline: "2020-01-01T00:00:00Z" };
    const svc = new RegistrationService(
      new FakeEventReader(expired) as any, repo as any, { generate: (p: string) => `qr-${p}` }
    );
    await expect(svc.register(student, "evt-1")).rejects.toBeInstanceOf(ValidationError);
  });
});

```

tests/registrationService.test.ts

3.6.9 Frontend Module

```
// frontend/js/events.js
```

```

// Frontend rule (Section 2.1 / 2.7): this module ONLY calls the backend
// API. No Supabase client, no business logic, no secrets.

const API_BASE = "/api";

async function fetchEvents(filters = {}) {
  const params = new URLSearchParams(filters);
  const res = await fetch(`${API_BASE}/events?${params}`, {
    headers: { Authorization: `Bearer ${getSessionToken()}` },
  });
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error ?? "Failed to load events");
  }
  const { data } = await res.json();
  return data;
}

async function registerForEvent(eventId) {
  const res = await fetch(`${API_BASE}/events/${eventId}/register`, {
    method: "POST",
    headers: { Authorization: `Bearer ${getSessionToken()}` },
  });
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error ?? "Registration failed");
  }
  return res.json();
}

function renderEventCard(event) {
  const card = document.createElement("article");
  card.className = "event-card";
  card.innerHTML = `
    <h3>${escapeHtml(event.title)}</h3>
    <p class="meta">${escapeHtml(event.venue)} · ${new Date(event.startTime).toLocaleString()}</p>
    <button data-event-id="${event.id}" class="register-btn">Register</button>
  `;
  return card;
}

function escapeHtml(str) {
  const div = document.createElement("div");
  div.textContent = str;
  return div.innerHTML;
}

function getSessionToken() {
  return window.localStorage.getItem("ce_session_token") ?? "";
}

document.addEventListener("DOMContentLoaded", async () => {
  const list = document.getElementById("event-list");
  try {
    const events = await fetchEvents();
    events.forEach(e => list.appendChild(renderEventCard(e)));
  } catch (err) {
    list.textContent = "Could not load events: " + err.message;
  }

  list.addEventListener("click", async (e) => {
    if (!e.target.matches(".register-btn")) return;
  });
});

```

```

    try {
      await registerForEvent(e.target.dataset.eventId);
      e.target.textContent = "Registered";
      e.target.disabled = true;
    } catch (err) {
      alert(err.message);
    }
  });
});
});

```

frontend/js/events.js — thin client, calls backend only, per Section 2.1/2.7.

3.7 SOLID Review Checklist for Increment 1

Principle	Where applied
SRP	domain/types.ts holds only shapes; routes/events.ts only parses HTTP; services hold only business rules; repositories only perform data access.
OCP	New EventFilters fields or a new QrCodeGenerator implementation can be added without modifying EventService's or RegistrationService's existing logic.
LSP	FakeEventRepository (tests) and SupabaseEventRepository both satisfy EventReader/EventWriter and are interchangeable by EventService.
ISP	EventReader and EventWriter are separate; a future read-only consumer (Increment 4 analytics) will depend only on EventReader.
DIP	EventService and RegistrationService depend on repository interfaces injected via constructor, never on the Supabase client directly.

3.8 Increment 1 Definition of Done

- All 8 test cases in Section 3.5 pass.
- Endpoints in the API contract (Section 2.6) tagged INC-1 are implemented and manually verified against Postman/curl.
- RLS policies from Section 2.4 covering events/registrations are applied to the Supabase project and verified with a non-privileged client.
- Frontend pages (index.html, login.html, register.html, organizer-dashboard.html create/edit view) are functional against the deployed backend.
- This increment is deployed to the free-tier environment (Section 2.10) and demonstrable end-to-end.

PHASE 4 — INCREMENT 2: APPROVAL WORKFLOW & NOTIFICATIONS

4.1 Increment Goal

Layer the Coordinator approval workflow and notification system on top of Increment 1 without modifying its existing services — only extending them, per the Open/Closed principle established in Section 2.9. Organizers can now submit drafts for approval; Coordinators can approve/reject with an audit trail; organizers and students receive notifications on key state changes. Corresponds to SRS requirements FR-3.4 and FR-4.1–FR-4.6.

4.2 Mini-Requirements (frozen scope for this increment)

ID	Requirement (abridged)
FR-3.4	Organizer submits a draft event for approval
FR-4.1 – FR-4.4	Coordinator approval queue, approve, reject with remarks, audit log
FR-4.5 – FR-4.6	Notify organizer on decision; notify students on material event changes

4.3 Increment Design Notes

This increment introduces two new abstractions rather than touching Increment 1's `EventService`: `NotificationChannel` (Section 4.5.1–4.5.2), allowing email and in-app delivery to be added/swapped independently, and `ApprovalRepository` (Section 4.5.3), a narrow, append-only interface separate from `EventWriter`. `ApprovalService` composes the existing `EventReader` from Increment 1 by reference — it does not duplicate event-fetching logic, demonstrating that the Increment 1 abstractions were designed correctly for reuse.

- Event status transitions (draft → pending → published/rejected) are now fully wired through `ApprovalService`.
- The events table's status column (Section 2.3) already models this; no schema migration is needed beyond what Increment 1 shipped.
- RLS policy `events_select_pending_coordinator` (Section 2.4) becomes active in this increment, gating the approval queue at the database level.

4.4 Frozen Test Plan for This Increment

Test ID	Scenario	Expected Result	Traces to
T2.1	Organizer submits their own draft event	Status becomes pending	FR-3.4
T2.2	Organizer submits another organizer's draft	403 Forbidden	Role-Permission Matrix
T2.3	Coordinator approves a pending event	Status becomes published; organizer notified	FR-4.2, FR-4.5
T2.4	Coordinator rejects without remarks	400 Validation error	FR-4.3

Test ID	Scenario	Expected Result	Traces to
T2.5	Coordinator rejects with remarks	Status becomes rejected; audit entry recorded; organizer notified	FR-4.3, FR-4.4, FR-4.5
T2.6	Organizer (non-coordinator) attempts to decide	403 Forbidden	Role-Permission Matrix
T2.7	Email channel fails but in-app channel succeeds	In-app notification still recorded; failure logged, not thrown	NFR-5 (resilience via OCP channel design)

4.5 Implementation

4.5.1 Notification Channel Interface (Open/Closed extension point)

```
// services/notification/notificationChannel.ts
// OCP in action: this interface is the extension point. Increment 2 ships
// EmailChannel and InAppChannel; a future SMS channel implements the same
// interface and is registered alongside the others – NotificationService
// (below) never needs to change.
```

```
export interface NotificationPayload {
  toUserId: string;
  subject: string;
  body: string;
}

export interface NotificationChannel {
  readonly name: string;
  send(payload: NotificationPayload): Promise<void>;
}
```

services/notification/notificationChannel.ts

4.5.2 Channel Implementations

```
// services/notification/channels.ts
// SRP: each channel's only job is delivering a message through one medium.
// EmailChannel wraps a provider adapter (Resend/SMTP-agnostic); InAppChannel
// simply writes a row the frontend polls/reads.

import { NotificationChannel, NotificationPayload } from "../notificationChannel";
import { SupabaseClient } from "@supabase/supabase-js";

export interface EmailProvider {
  sendEmail(to: string, subject: string, body: string): Promise<void>;
}

export class EmailChannel implements NotificationChannel {
  readonly name = "email";
  constructor(private readonly provider: EmailProvider, private readonly resolveEmail: (userId: string) => Promise<string>) {}

  async send(payload: NotificationPayload): Promise<void> {
    const email = await this.resolveEmail(payload.toUserId);
    await this.provider.sendEmail(email, payload.subject, payload.body);
  }
}
```

```
export class InAppChannel implements NotificationChannel {
  readonly name = "in_app";
  constructor(private readonly db: SupabaseClient) {}

  async send(payload: NotificationPayload): Promise<void> {
    const { error } = await this.db.from("notifications").insert({
      user_id: payload.toUserId, subject: payload.subject, body: payload.body, read: false,
    });
    if (error) throw error;
  }
}
```

services/notification/channels.ts

4.5.3 Notification Service (fan-out)

```
// services/notification/notificationService.ts
// SRP + DIP: this service's only job is fanning a single logical
// notification out across every registered channel. It knows nothing about
// email providers or database tables – those live behind the
// NotificationChannel interface, injected as an array.

import { NotificationChannel, NotificationPayload } from "../notificationChannel";

export class NotificationService {
  constructor(private readonly channels: NotificationChannel[]) {}

  async notify(payload: NotificationPayload): Promise<void> {
    // Failures in one channel must not block others (e.g. email provider
    // down should not prevent the in-app notification from being recorded).
    const results = await Promise.allSettled(this.channels.map(c => c.send(payload)));
    results.forEach((r, i) => {
      if (r.status === "rejected") {
        console.error(`Notification channel "${this.channels[i].name}" failed:`, r.reason);
      }
    });
  }
}
```

services/notification/notificationService.ts

4.5.4 Approval Repository Interface & Service

```
// repositories/interfaces.ts (additions for Increment 2)
// ISP: a new narrow interface for the append-only audit log, kept separate
// from EventWriter so nothing but the approval flow can write decisions.

import { EventStatus } from "../domain/types";

export interface ApprovalRecord {
  id: string; eventId: string; coordinatorId: string;
  decision: "approved" | "rejected"; remarks?: string; decidedAt: string;
}

export interface ApprovalRepository {
  recordDecision(eventId: string, coordinatorId: string, decision: "approved" | "rejected",
  remarks?: string): Promise<ApprovalRecord>;
  findPendingEvents(): Promise<{ id: string; title: string; organizerId: string }[]>;
}

export interface EventStatusWriter {
  setStatus(eventId: string, status: EventStatus): Promise<void>;
}
```

```

}

// services/approvalService.ts
// SRP: owns approval-workflow rules only. Composes EventReader (read-only,
// per ISP from Increment 1), EventStatusWriter, ApprovalRepository, and
// NotificationService – depends on abstractions throughout (DIP).

import { EventReader } from "../repositories/interfaces";
import { NotificationService } from "../notification/notificationService";
import { Profile } from "../domain/types";
import { ForbiddenError, NotFoundError, ValidationError } from "../domain/errors";

export class ApprovalService {
  constructor(
    private readonly events: EventReader,
    private readonly statusWriter: EventStatusWriter,
    private readonly approvals: ApprovalRepository,
    private readonly notifications: NotificationService
  ) {}

  async submitForApproval(actor: Profile, eventId: string): Promise<void> {
    const event = await this.events.findById(eventId);
    if (!event) throw new NotFoundError("Event");
    if (event.organizerId !== actor.id && actor.role !== "super_admin") {
      throw new ForbiddenError("You do not own this event");
    }
    if (event.status !== "draft") {
      throw new ValidationError("Only draft events can be submitted for approval");
    }
    await this.statusWriter.setStatus(eventId, "pending"); // FR-3.4
  }

  async listPending() {
    return this.approvals.findPendingEvents(); // FR-4.1
  }

  async decide(actor: Profile, eventId: string, decision: "approved" | "rejected", remarks?:
string): Promise<void> {
    if (actor.role !== "coordinator" && actor.role !== "super_admin") {
      throw new ForbiddenError("Only coordinators may approve or reject events");
    }
    if (decision === "rejected" && !remarks?.trim()) {
      throw new ValidationError("Remarks are mandatory when rejecting an event"); // FR-4.3
    }
    const event = await this.events.findById(eventId);
    if (!event) throw new NotFoundError("Event");
    if (event.status !== "pending") {
      throw new ValidationError("Only pending events can be approved or rejected");
    }

    await this.approvals.recordDecision(eventId, actor.id, decision, remarks); // FR-4.4, audit
    trail
    await this.statusWriter.setStatus(eventId, decision === "approved" ? "published" :
"rejected"); // FR-4.2

    await this.notifications.notify({
      toUserId: event.organizerId,
      subject: decision === "approved" ? `Event "${event.title}" approved` : `Event
"${event.title}" rejected`,
      body: decision === "approved"
        ? "Your event is now published and visible to students."
        : `Your event was rejected. Coordinator remarks: ${remarks}`,
    });
  }
}

```

```

    }); // FR-4.5
  }
}

```

repositories/interfaces.ts additions + services/approvalService.ts

4.5.5 Unit Tests

```

// tests/approvalService.test.ts

import { describe, it, expect, beforeEach } from "vitest";
import { ApprovalService } from "../services/approvalService";
import { NotificationService } from "../services/notification/notificationService";
import { EventRecord, Profile } from "../domain/types";
import { ForbiddenError, ValidationError } from "../domain/errors";

class FakeEventReader {
  constructor(private event: EventRecord) {}
  async findById(id: string) { return id === this.event.id ? this.event : null; }
}
class FakeStatusWriter {
  calls: { eventId: string; status: string }[] = [];
  async setStatus(eventId: string, status: string) { this.calls.push({ eventId, status }); }
}
class FakeApprovalRepo {
  decisions: any[] = [];
  async recordDecision(eventId: string, coordinatorId: string, decision: string, remarks?: string) {
    const rec = { id: "a1", eventId, coordinatorId, decision, remarks, decidedAt: new
Date().toISOString() };
    this.decisions.push(rec);
    return rec;
  }
  async findPendingEvents() { return []; }
}
class FakeNotificationService extends NotificationService {
  sent: any[] = [];
  constructor() { super([]); }
  async notify(payload: any) { this.sent.push(payload); }
}

const coordinator: Profile = { id: "coord-1", fullName: "Dr. Rao", role: "coordinator" };
const organizer: Profile = { id: "org-1", fullName: "Coding Club", role: "organizer" };
const pendingEvent: EventRecord = {
  id: "evt-1", organizerId: "org-1", title: "Hackathon", description: "", category: "Tech",
  venue: "Hall", startTime: "2027-01-01T09:00:00Z", endTime: "2027-01-01T18:00:00Z",
  registrationDeadline: "2026-12-30T00:00:00Z", capacity: 100, status: "pending", createdAt: "",
};

describe("ApprovalService.decide", () => {
  let statusWriter: FakeStatusWriter, approvals: FakeApprovalRepo, notifications:
FakeNotificationService, service: ApprovalService;
  beforeEach(() => {
    statusWriter = new FakeStatusWriter();
    approvals = new FakeApprovalRepo();
    notifications = new FakeNotificationService();
    service = new ApprovalService(new FakeEventReader(pendingEvent) as any, statusWriter,
approvals as any, notifications);
  });

  it("publishes the event and notifies the organizer on approval", async () => {
    await service.decide(coordinator, "evt-1", "approved");
    expect(statusWriter.calls).toContainEqual({ eventId: "evt-1", status: "published" });
    expect(notifications.sent[0].toUserId).toBe("org-1");
  });
}

```

```

});

it("rejects a decision from a non-coordinator", async () => {
  await expect(service.decide(organizer, "evt-1",
    "approved")).rejects.toBeInstanceOf(ForbiddenError);
});

it("requires remarks when rejecting", async () => {
  await expect(service.decide(coordinator, "evt-1",
    "rejected")).rejects.toBeInstanceOf(ValidationError);
});

it("records an audit entry for every decision", async () => {
  await service.decide(coordinator, "evt-1", "rejected", "Venue unavailable");
  expect(approvals.decisions).toHaveLength(1);
  expect(approvals.decisions[0].remarks).toBe("Venue unavailable");
});
});

```

tests/approvalService.test.ts

4.6 SOLID Review Checklist for Increment 2

Principle	Where applied
SRP	ApprovalService owns workflow rules only; NotificationService owns fan-out only; each channel owns one delivery medium.
OCP	A new SMS/push channel can be added by implementing NotificationChannel — no existing class is modified.
LSP	EmailChannel and InAppChannel are interchangeable wherever NotificationChannel is expected; FakeNotificationService in tests substitutes cleanly.
ISP	ApprovalRepository is separate from EventWriter/EventReader — coordinators' audit-writing capability is not bundled into unrelated interfaces.
DIP	ApprovalService depends on EventReader, EventStatusWriter, ApprovalRepository, and NotificationService — all abstractions, all injected.

4.7 Increment 2 Definition of Done

- All 7 test cases in Section 4.4 pass, alongside all Increment 1 tests still passing (regression check).
- coordinator-queue.html frontend page lists pending events and posts decisions via the API only.
- Audit entries are verified immutable (no UPDATE/DELETE route exists for event_approvals).
- Notifications table and email delivery are verified end-to-end in the deployed environment.

PHASE 5 — INCREMENT 3: ATTENDANCE (QR CHECK-IN) & CERTIFICATES

5.1 Increment Goal

Add venue check-in via QR code and automatic certificate generation on event completion, building on the QR code already generated at registration time in Increment 1 (Section 3.6.1, `Registration.qrCode`). Corresponds to SRS requirements FR-5.1–FR-5.5.

5.2 Mini-Requirements (frozen scope for this increment)

ID	Requirement (abridged)
FR-5.1	Unique QR code per registration (already generated in Increment 1; this increment consumes it)
FR-5.2 – FR-5.3	Organizer marks attendance via QR; a QR code cannot be reused
FR-5.4 – FR-5.5	Auto-generate PDF certificates on event completion; students can download theirs

5.3 Increment Design Notes

Two new abstractions are introduced, both following the Open/Closed pattern established in Increment 2's `NotificationChannel`: `CertificateGenerator` (Section 5.5.1–5.5.2), so the PDF library can be swapped without touching `AttendanceService`, and `StorageUploader`, so certificate storage can move from Supabase Storage to any other blob store without a service-layer change. `AttendanceService` composes `EventReader` and `EventStatusWriter` from Increments 1–2 rather than re-implementing event lookups.

- certificates table (Section 2.3) is populated only after `completeEvent` runs, and only for students marked attended — satisfying FR-5.4's dependency on attendance, not merely registration.
- The idempotency guard (`existsFor` check) means `completeEvent` can be safely retried if the request fails partway, a resilience property directly reviewable in code.

5.4 Frozen Test Plan for This Increment

Test ID	Scenario	Expected Result	Traces to
T3.1	Organizer checks in a valid, unused QR code	Registration marked attended	FR-5.2
T3.2	Organizer checks in an already-used QR code	400 Validation error	FR-5.3
T3.3	Organizer checks in an unknown/invalid QR code	404 Not found	FR-5.2
T3.4	Non-owning organizer attempts check-in	403 Forbidden	Role-Permission Matrix
T3.5	Organizer completes an event with attendees	Event status = completed; one certificate generated per attendee	FR-5.4

Test ID	Scenario	Expected Result	Traces to
T3.6	completeEvent is called twice for the same event	No duplicate certificates created (idempotency)	NFR-5
T3.7	Student downloads their own certificate	PDF URL returned only for that student's own registration	FR-5.5, NFR-1

5.5 Implementation

5.5.1 Certificate Generator Interface

```
// services/certificate/certificateGenerator.ts
// OCP: the PDF rendering technology (e.g. pdf-lib, Puppeteer-to-PDF) sits
// behind this interface. CertificateService (below) never depends on a
// specific PDF library – swapping renderers touches only the
// implementation, not the service that calls it.
```

```
export interface CertificateData {
  studentName: string;
  eventTitle: string;
  eventDate: string;
  organizerClub: string;
}
```

```
export interface CertificateGenerator {
  generate(data: CertificateData): Promise<Buffer>; // returns PDF bytes
}
```

services/certificate/certificateGenerator.ts

5.5.2 Certificate Generator Implementation

```
// services/certificate/pdfLibCertificateGenerator.ts
// SRP: this class's only job is rendering CertificateData into PDF bytes
// using pdf-lib. No storage, no DB writes – those belong to
// CertificateService and the repository layer respectively.
```

```
import { PDFDocument, StandardFonts, rgb } from "pdf-lib";
import { CertificateGenerator, CertificateData } from "../certificateGenerator";
```

```
export class PdfLibCertificateGenerator implements CertificateGenerator {
  async generate(data: CertificateData): Promise<Buffer> {
    const doc = await PDFDocument.create();
    const page = doc.addPage([842, 595]); // A4 landscape
    const font = await doc.embedFont(StandardFonts.HelveticaBold);
    const bodyFont = await doc.embedFont(StandardFonts.Helvetica);

    page.drawText("Certificate of Participation", {
      x: 180, y: 480, size: 28, font, color: rgb(0.12, 0.22, 0.4),
    });
    page.drawText(`This certifies that ${data.studentName}`, {
      x: 180, y: 400, size: 16, font: bodyFont,
    });
    page.drawText(`participated in "${data.eventTitle}"`, {
      x: 180, y: 370, size: 16, font: bodyFont,
    });
    page.drawText(`organized by ${data.organizerClub} on ${data.eventDate}`, {
      x: 180, y: 340, size: 14, font: bodyFont, color: rgb(0.3, 0.3, 0.3),
    });
  }
}
```

```

    const bytes = await doc.save();
    return Buffer.from(bytes);
  }
}

```

services/certificate/pdfLibCertificateGenerator.ts

5.5.3 Attendance Service

```

// services/attendanceService.ts
// SRP: owns attendance/check-in and completion rules. Depends on
// RegistrationRepository (Increment 1, extended), EventReader/
// EventStatusWriter (Increment 1/2), CertificateGenerator (abstraction,
// DIP), and a StorageUploader abstraction for persisting the PDF.

import { EventReader } from "../repositories/interfaces";
import { EventStatusWriter } from "../repositories/interfaces";
import { CertificateGenerator } from "../certificate/certificateGenerator";
import { Profile } from "../domain/types";
import { ForbiddenError, NotFoundError, ValidationError } from "../domain/errors";

export interface AttendanceRepository {
  findByQrCode(qrCode: string): Promise<{ id: string; eventId: string; studentId: string;
  attended: boolean } | null>;
  markAttended(registrationId: string): Promise<void>;
  findAttendedForEvent(eventId: string): Promise<{ registrationId: string; studentId: string;
  studentName: string }[]>;
}

export interface CertificateRepository {
  create(registrationId: string, pdfUrl: string): Promise<void>;
  existsFor(registrationId: string): Promise<boolean>;
}

export interface StorageUploader {
  upload(path: string, data: Buffer, contentType: string): Promise<string>; // returns public URL
}

export class AttendanceService {
  constructor(
    private readonly events: EventReader,
    private readonly statusWriter: EventStatusWriter,
    private readonly attendance: AttendanceRepository,
    private readonly certificates: CertificateRepository,
    private readonly certGenerator: CertificateGenerator,
    private readonly storage: StorageUploader
  ) {}

  async checkIn(actor: Profile, eventId: string, qrCode: string): Promise<void> {
    const event = await this.events.findById(eventId);
    if (!event) throw new NotFoundError("Event");
    if (event.organizerId !== actor.id && actor.role !== "super_admin") {
      throw new ForbiddenError("Only the organizing club may check in attendees");
    }

    const registration = await this.attendance.findByQrCode(qrCode);
    if (!registration || registration.eventId !== eventId) {
      throw new NotFoundError("Registration");
    }
    if (registration.attended) {
      throw new ValidationError("This QR code has already been used for check-in"); // FR-5.3
    }
  }
}

```



```

    await this.attendance.markAttended(registration.id); // FR-5.2
  }

  async completeEvent(actor: Profile, eventId: string): Promise<void> {
    const event = await this.events.findById(eventId);
    if (!event) throw new NotFoundError("Event");
    if (event.organizerId !== actor.id && actor.role !== "super_admin") {
      throw new ForbiddenError("Only the organizing club may complete this event");
    }

    await this.statusWriter.setStatus(eventId, "completed");

    const attendees = await this.attendance.findAttendedForEvent(eventId);
    for (const attendee of attendees) {
      if (await this.certificates.existsFor(attendee.registrationId)) continue; // idempotency
    }
    const pdf = await this.certGenerator.generate({
      studentName: attendee.studentName,
      eventTitle: event.title,
      eventDate: new Date(event.startTime).toLocaleDateString(),
      organizerClub: event.organizerId,
    });
    const url = await this.storage.upload(
      `certificates/${attendee.registrationId}.pdf`, pdf, "application/pdf"
    );
    await this.certificates.create(attendee.registrationId, url); // FR-5.4
  }
}

```

services/attendanceService.ts

5.5.4 Unit Tests

```

// tests/attendanceService.test.ts

import { describe, it, expect, beforeEach, vi } from "vitest";
import { AttendanceService } from "../services/attendanceService";
import { EventRecord, Profile } from "../domain/types";
import { ForbiddenError, NotFoundError, ValidationError } from "../domain/errors";

const organizer: Profile = { id: "org-1", fullName: "Coding Club", role: "organizer" };
const event: EventRecord = {
  id: "evt-1", organizerId: "org-1", title: "Hackathon", description: "", category: "Tech",
  venue: "Hall", startTime: "2026-09-01T09:00:00Z", endTime: "2026-09-01T18:00:00Z",
  registrationDeadline: "2026-08-30T00:00:00Z", capacity: 100, status: "published", createdAt: "",
};

function buildService(overrides: Partial<any> = {}) {
  const events = { findById: vi.fn().mockResolvedValue(event) };
  const statusWriter = { setStatus: vi.fn().mockResolvedValue(undefined) };
  const attendance = {
    findByQrCode: vi.fn().mockResolvedValue({ id: "reg-1", eventId: "evt-1", studentId: "stu-1",
    attended: false }),
    markAttended: vi.fn().mockResolvedValue(undefined),
    findAttendedForEvent: vi.fn().mockResolvedValue([]),
    ...overrides.attendance,
  };
  const certificates = { create: vi.fn(), existsFor: vi.fn().mockResolvedValue(false) };
  const certGenerator = { generate: vi.fn().mockResolvedValue(Buffer.from("pdf")) };
  const storage = { upload: vi.fn().mockResolvedValue("https://storage/cert.pdf") };
  return new AttendanceService(events as any, statusWriter as any, attendance as any, certificates
  as any, certGenerator as any, storage as any);
}

```

```

}

describe("AttendanceService.checkIn", () => {
  it("marks a valid, unused QR code as attended", async () => {
    const service = buildService();
    await expect(service.checkIn(organizer, "evt-1", "qr-1")).resolves.toBeUndefined();
  });

  it("rejects check-in by a non-owning organizer", async () => {
    const other: Profile = { id: "org-2", fullName: "Other Club", role: "organizer" };
    const service = buildService();
    await expect(service.checkIn(other, "evt-1", "qr-1")).rejects.toBeInstanceOf(ForbiddenError);
  });

  it("rejects a QR code already marked attended (FR-5.3)", async () => {
    const service = buildService({
      attendance: { findByQrCode: vi.fn().mockResolvedValue({ id: "reg-1", eventId: "evt-1",
studentId: "stu-1", attended: true }) },
    });
    await expect(service.checkIn(organizer, "evt-1", "qr-
1")).rejects.toBeInstanceOf(ValidationError);
  });

  it("rejects an unknown QR code", async () => {
    const service = buildService({ attendance: { findByQrCode: vi.fn().mockResolvedValue(null) }
});
    await expect(service.checkIn(organizer, "evt-1", "bad-
qr")).rejects.toBeInstanceOf(NotFoundError);
  });
});

describe("AttendanceService.completeEvent", () => {
  it("generates a certificate for each attended student exactly once (idempotent)", async () => {
    const findAttendedForEvent = vi.fn().mockResolvedValue([
      { registrationId: "reg-1", studentId: "stu-1", studentName: "A Student" },
    ]);
    const existsFor = vi.fn().mockResolvedValue(false);
    const create = vi.fn();
    const service = buildService({ attendance: { findAttendedForEvent } });
    (service as any).certificates = { existsFor, create };
    await service.completeEvent(organizer, "evt-1");
    expect(findAttendedForEvent).toHaveBeenCalled("evt-1");
  });
});

```

tests/attendanceService.test.ts

5.6 SOLID Review Checklist for Increment 3

Principle	Where applied
SRP	PdfLibCertificateGenerator only renders PDFs; AttendanceService only orchestrates check-in/completion rules; storage concerns are isolated behind StorageUploader.
OCP	Switching from pdf-lib to another PDF engine, or from Supabase Storage to S3, requires a new class implementing the existing interface — zero changes to AttendanceService.
LSP	Any StorageUploader implementation is substitutable; tests use an in-memory fake with the identical method signature as the real Supabase Storage uploader.

Principle	Where applied
ISP	AttendanceRepository (check-in/query) and CertificateRepository (cert persistence) are kept separate rather than merged into one large repository.
DIP	AttendanceService's constructor takes six abstractions, none of which is a concrete Supabase or pdf-lib type.

5.7 Increment 3 Definition of Done

- All 7 test cases in Section 5.4 pass, alongside all Increment 1–2 tests (regression check).
- QR check-in is verified against a physically scannable code rendered on the student's my-events.html page.
- Certificates are downloadable end-to-end from Supabase Storage in the deployed environment.

PHASE 6 — INCREMENT 4: ADMIN ANALYTICS & FEEDBACK

6.1 Increment Goal

Deliver the final planned increment: a Super Admin analytics dashboard aggregating data across all prior increments (events, registrations, approvals, attendance), and a student feedback loop on completed events. This closes the SRS requirement set (FR-6.1–FR-6.5) and completes the four-increment plan defined in the Feasibility Study (Section 0.3.4).

6.2 Mini-Requirements (frozen scope for this increment)

ID	Requirement (abridged)
FR-6.1 – FR-6.2	Super Admin manages users/roles and moderates any event
FR-6.3	Analytics dashboard: events per period, registrations per club, approval turnaround, attendance rate
FR-6.4 – FR-6.5	Students submit post-event feedback; aggregated ratings shown to organizer/admin

6.3 Increment Design Notes

This increment deliberately introduces read-only, aggregation-focused interfaces (AnalyticsRepository, Section 6.5.1) that never expose write methods — nothing in the analytics pathway can mutate event/registration data, which is both an ISP demonstration and a safety property worth calling out in review. FeedbackService reuses the RegistrationReader pattern established across Increments 1–3: a narrow read interface rather than depending on the full RegistrationRepository.

- Approval turnaround time is computed from events.created_at (or submission timestamp) to event_approvals.decided_at — both already captured by the Increment 1–2 schema, requiring no new columns.
- Attendance rate reuses the registrations.attended flag from Increment 3, and feedback reuses the feedback table defined in Section 2.3.
- No further schema migrations are required for this increment beyond the tables already defined in Section 2.3 — evidence that the up-front data model design (Section 2.3) correctly anticipated all four increments.

6.4 Frozen Test Plan for This Increment

Test ID	Scenario	Expected Result	Traces to
T4.1	Super Admin requests the analytics dashboard	Snapshot with all four metric categories returned	FR-6.3
T4.2	Organizer or Coordinator requests the analytics dashboard	403 Forbidden	Role-Permission Matrix
T4.3	Student submits feedback after attending a completed event	Feedback recorded	FR-6.4
T4.4	Student submits a rating outside 1–5	400 Validation error	FR-6.4

Test ID	Scenario	Expected Result	Traces to
T4.5	Student submits feedback for an event not yet completed	400 Validation error	FR-6.4
T4.6	Student submits feedback twice for the same registration	400 Validation error	FR-6.4
T4.7	Organizer views aggregated feedback for their own event	Average rating and count returned	FR-6.5

6.5 Implementation

6.5.1 Analytics Repository Interface & Service

```
// repositories/analyticsRepository.ts
// SRP + ISP: a read-only, aggregation-focused interface, entirely separate
// from EventReader/EventWriter/RegistrationRepository. Analytics consumers
// never need write access, so this interface never exposes any.

export interface AnalyticsSnapshot {
  totalEvents: number;
  registrationsByClub: { club: string; count: number }[];
  averageApprovalTurnaroundHours: number;
  attendanceRateByEvent: { eventId: string; title: string; rate: number }[];
}

export interface AnalyticsRepository {
  getSnapshot(fromDate?: string, toDate?: string): Promise<AnalyticsSnapshot>;
}

// services/analyticsService.ts
// SRP: thin orchestration only – authorization check, then delegate to the
// repository. All aggregation SQL/logic lives in the repository
// implementation (Supabase-backed), keeping this service trivially testable.

import { Profile } from "../domain/types";
import { ForbiddenError } from "../domain/errors";

export class AnalyticsService {
  constructor(private readonly analytics: AnalyticsRepository) {}

  async getDashboard(actor: Profile, fromDate?: string, toDate?: string):
  Promise<AnalyticsSnapshot> {
    if (actor.role !== "super_admin") {
      throw new ForbiddenError("Only the super admin may view analytics"); // FR-6.1/6.3, Role-
      Permission Matrix
    }
    return this.analytics.getSnapshot(fromDate, toDate);
  }
}
```

repositories/analyticsRepository.ts + services/analyticsService.ts

6.5.2 Feedback Service

```
// services/feedbackService.ts
// SRP: owns feedback submission rules only. Depends on a narrow
// FeedbackRepository and RegistrationReader (not the full Registration
// repository), per ISP.
```

```

export interface FeedbackRepository {
  create(registrationId: string, rating: number, comment?: string): Promise<void>;
  existsFor(registrationId: string): Promise<boolean>;
  aggregateForEvent(eventId: string): Promise<{ averageRating: number; count: number }>;
}

export interface RegistrationReader {
  findById(registrationId: string): Promise<{ id: string; studentId: string; eventId: string;
  attended: boolean } | null>;
}

export interface EventStatusReader {
  isCompleted(eventId: string): Promise<boolean>;
}

import { Profile } from "../domain/types";
import { ForbiddenError, NotFoundError, ValidationError } from "../domain/errors";

export class FeedbackService {
  constructor(
    private readonly registrations: RegistrationReader,
    private readonly events: EventStatusReader,
    private readonly feedback: FeedbackRepository
  ) {}

  async submit(actor: Profile, registrationId: string, rating: number, comment?: string):
  Promise<void> {
    if (rating < 1 || rating > 5) {
      throw new ValidationError("Rating must be between 1 and 5");
    }
    const registration = await this.registrations.findById(registrationId);
    if (!registration) throw new NotFoundError("Registration");
    if (registration.studentId !== actor.id) {
      throw new ForbiddenError("You may only submit feedback for your own registration");
    }
    if (!registration.attended) {
      throw new ValidationError("Feedback can only be submitted after attending the event");
    }
    const completed = await this.events.isCompleted(registration.eventId);
    if (!completed) throw new ValidationError("Feedback can only be submitted after the event is
    completed"); // FR-6.4

    const alreadySubmitted = await this.feedback.existsFor(registrationId);
    if (alreadySubmitted) throw new ValidationError("Feedback already submitted for this
    registration");

    await this.feedback.create(registrationId, rating, comment);
  }

  async getEventSummary(eventId: string) {
    return this.feedback.aggregateForEvent(eventId); // FR-6.5
  }
}

```

services/feedbackService.ts

6.5.3 Unit Tests

```

// tests/analyticsService.test.ts and feedbackService.test.ts (combined for brevity)

import { describe, it, expect } from "vitest";

```

```

import { AnalyticsService } from "../services/analyticsService";
import { FeedbackService } from "../services/feedbackService";
import { Profile } from "../domain/types";
import { ForbiddenError, ValidationError } from "../domain/errors";

const admin: Profile = { id: "admin-1", fullName: "Registrar", role: "super_admin" };
const organizer: Profile = { id: "org-1", fullName: "Coding Club", role: "organizer" };
const student: Profile = { id: "stu-1", fullName: "A Student", role: "student" };

describe("AnalyticsService.getDashboard", () => {
  const fakeSnapshot = { totalEvents: 10, registrationsByClub: [], averageApprovalTurnaroundHours:
4, attendanceRateByEvent: [] };
  const repo = { getSnapshot: async () => fakeSnapshot };

  it("returns the dashboard snapshot for a super admin", async () => {
    const service = new AnalyticsService(repo);
    const result = await service.getDashboard(admin);
    expect(result.totalEvents).toBe(10);
  });

  it("rejects access from a non-admin role", async () => {
    const service = new AnalyticsService(repo);
    await expect(service.getDashboard(organizer)).rejects.toBeInstanceOf(ForbiddenError); // FR-
6.1
  });
});

describe("FeedbackService.submit", () => {
  const attendedReg = { id: "reg-1", studentId: "stu-1", eventId: "evt-1", attended: true };
  const registrations = { findById: async () => attendedReg };
  const events = { isCompleted: async () => true };
  const feedback = { create: async () => {}, existsFor: async () => false, aggregateForEvent:
async () => ({ averageRating: 0, count: 0 }) };

  it("accepts valid feedback after attendance and completion", async () => {
    const service = new FeedbackService(registrations, events, feedback);
    await expect(service.submit(student, "reg-1", 5, "Great event")).resolves.toBeUndefined();
  });

  it("rejects a rating outside 1-5", async () => {
    const service = new FeedbackService(registrations, events, feedback);
    await expect(service.submit(student, "reg-1", 7)).rejects.toBeInstanceOf(ValidationError);
  });

  it("rejects feedback for an event that is not yet completed", async () => {
    const service = new FeedbackService(registrations, { isCompleted: async () => false },
feedback);
    await expect(service.submit(student, "reg-1", 4)).rejects.toBeInstanceOf(ValidationError);
// FR-6.4
  });

  it("rejects duplicate feedback submission", async () => {
    const service = new FeedbackService(registrations, events, { ...feedback, existsFor: async ()
=> true });
    await expect(service.submit(student, "reg-1", 4)).rejects.toBeInstanceOf(ValidationError);
  });
});

```

tests/analyticsService.test.ts + feedbackService.test.ts

6.6 SOLID Review Checklist for Increment 4

Principle	Where applied
SRP	AnalyticsService only authorizes + delegates; all aggregation logic is isolated in the repository implementation, not mixed into the service.
OCP	New dashboard metrics are added by extending AnalyticsSnapshot and the repository query, without changing AnalyticsService's authorization logic.
LSP	Any AnalyticsRepository/FeedbackRepository implementation (Supabase-backed or an in-memory fake for tests) is substitutable without changing service behavior.
ISP	AnalyticsRepository is read-only by design; RegistrationReader exposes only findById, not the full RegistrationRepository surface from Increment 1.
DIP	Both AnalyticsService and FeedbackService depend exclusively on interfaces passed into their constructors.

6.7 Increment 4 Definition of Done

- All 7 test cases in Section 6.4 pass, alongside the full regression suite from Increments 1–3.
- admin-dashboard.html renders all four metric categories from the deployed backend.
- Feedback submission is verified end-to-end from the student's my-events.html page.
- This marks the final planned increment; the project proceeds to overall system testing (Section 7) and deployment (Section 8).

PHASE 7 — OVERALL TESTING STRATEGY

7.1 Testing Levels

- Unit tests: each service tested in isolation against fake/in-memory implementations of its dependency interfaces (all four increments, Sections 3.6.8, 4.5.5, 5.5.4, 6.5.3) — this is only possible because of the Dependency Inversion applied throughout (Section 2.9).
- Integration tests: routes exercised against a real (local/staging) Supabase instance to confirm RLS policies (Section 2.4) behave identically to the service-layer checks — verifying the defense-in-depth design actually holds under real database enforcement.
- Manual/exploratory testing: each increment's Definition of Done (Sections 3.8, 4.7, 5.7, 6.7) includes manual verification via the deployed frontend before the increment is considered complete.
- Regression testing: the full test suite from all completed increments is re-run before starting the next increment's implementation, catching any interface drift early.

7.2 Requirements Traceability Matrix

Every functional requirement from the SRS (Section 1.5) is mapped to the test case(s) that verify it, closing the loop from requirement to design to code to test — the structural backbone that makes this an auditable engineering document rather than a collection of code snippets.

Requirement Group	Requirement IDs	Verified by Test IDs
Authentication & Authorization	FR-1.1 – FR-1.4	T1.8 and role checks embedded in T1.2, T2.2, T2.6, T4.2
Event Browsing & Registration	FR-2.1 – FR-2.6	T1.4, T1.5, T1.6, T1.7
Event Creation & Management	FR-3.1 – FR-3.4	T1.1, T1.2, T1.3, T2.1, T2.2
Approval Workflow	FR-4.1 – FR-4.6	T2.3, T2.4, T2.5, T2.6, T2.7
Attendance & Certification	FR-5.1 – FR-5.5	T3.1 – T3.7
Administration & Analytics	FR-6.1 – FR-6.5	T4.1 – T4.7

7.3 Non-Functional Requirement Verification

NFR	Verification Method
NFR-1 (Security)	RLS policy tests using a non-privileged Supabase client (Section 2.4); backend authorization tests (T1.2, T2.2, T2.6, T4.2).
NFR-2 (Performance)	Manual timing of GET /api/events under representative filter combinations against the deployed Render instance.
NFR-3 (Availability)	Documented cold-start behavior accepted per Risk Register (Section 0.7); not subject to automated SLA testing given free-tier hosting.
NFR-5 (Maintainability)	SOLID review checklists per increment (Sections 3.7, 4.6, 5.6, 6.6); each increment adds code without modifying prior services, confirmed by passing regression suites.

NFR	Verification Method
NFR-7 (Data Integrity)	T1.5, T1.6 confirm application-level enforcement; the database constraints and trigger in Section 2.3 are additionally verified by attempting direct SQL inserts that violate capacity/uniqueness.
NFR-8 (Auditability)	T2.5 confirms audit entries are recorded; absence of any UPDATE/DELETE route on event_approvals is confirmed by code review (Section 4.7).

PHASE 8 — DEPLOYMENT PLAN

8.1 Environments

Environment	Purpose	Hosting
Local	Development, unit tests, manual verification before each increment's Definition of Done	Node.js locally + Supabase local emulator or a dev Supabase project
Staging	Integration testing against real Supabase, pre-demo verification	Render (backend, free tier) + Supabase (dev project)
Production	Final deployed system for presentation and grading	Vercel (frontend) + Render (backend) + Supabase (production project)

8.2 Configuration Management

Secrets are never committed to source control. Each environment holds its own copy of the variables below, configured through the hosting provider's environment-variable settings (Vercel/Render dashboards) rather than in-repo .env files.

```
# .env (backend) — never committed to source control
SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_ANON_KEY=<anon-key>           # used for RLS-scoped, user-context queries
SUPABASE_SERVICE_ROLE_KEY=<service-key> # backend-only; never sent to the frontend
JWT_SECRET=<supabase-jwt-secret>        # for verifying incoming session tokens
EMAIL_PROVIDER_API_KEY=<provider-key>
PORT=4000
NODE_ENV=production
```

.env.example — committed as a template only; real values are set per environment in the hosting dashboards.

8.3 Deployment Steps

8. Provision a Supabase project; run `schema.sql` and `rls_policies.sql` (Sections 2.3–2.4) via the Supabase SQL editor or CLI migrations.
9. Configure Supabase Auth (email/password provider) and set the JWT secret used for backend verification.
10. Deploy the Node.js/TypeScript backend to Render as a web service, setting the environment variables from Section 8.2.
11. Deploy the static frontend (HTML/CSS/JS) to Vercel, pointing its `fetch()` calls at the Render backend's public URL.
12. Run the full automated test suite (Section 7) against the staging environment before promoting to production.
13. Smoke-test each increment's Definition of Done (Sections 3.8, 4.7, 5.7, 6.7) against the live production URL.

8.4 Continuous Integration (recommended)

Tests are run automatically on every push, catching regressions between increments before manual smoke-testing (Section 8.3, step 6).

```
# .github/workflows/ci.yml (conceptual – run tests on every push)
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run typecheck
      - run: npm run test -- --run
```

.github/workflows/ci.yml — conceptual CI pipeline.

8.5 Maintenance Plan

- Corrective maintenance: defects discovered post-deployment are triaged against the Role-Permission Matrix (Section 1.8) and Risk Register (Section 0.7) to assess severity before patching.
- Adaptive maintenance: Supabase/Render/Vercel free-tier limits (Section 0.3.2) are monitored; migration to paid tiers is the documented response if usage grows beyond a single college.
- Perimeter for future increments: payment processing, native mobile apps, and multi-tenancy (Section 0.5, Out of Scope) are the natural next increments if the project continues beyond this academic submission, following the same Incremental model and SOLID-based architecture established here.

PHASE 9 — PRESENTATION SUMMARY

9.1 Purpose of This Section

This section condenses the full document into the narrative arc suitable for a viva/defense presentation, referencing the detailed sections above for supporting evidence. It is structured as a suggested slide-by-slide outline.

9.2 Suggested Slide Outline

Slide	Content	Source Section
1	Title: CampusEvents — College Event Management Portal	Cover
2	Problem Statement — fragmented, manual event management	0.2
3	Feasibility Summary — technical, economic, operational, schedule (all green)	0.3
4	Why the Incremental Model (vs. Waterfall/Prototyping/Spiral)	0.4
5	Scope — in scope vs. explicitly deferred	0.5
6	Architecture Diagram — three-tier, frontend/backend/data separation	2.1–2.2
7	ER Diagram / Schema Summary	2.3
8	RLS-First Security Model	2.4
9	SOLID Principles — the standing rule set for all code	2.9
10	Increment 1 Demo — Auth, Browse, Register, Create	Phase 3
11	Increment 2 Demo — Approval Workflow & Notifications	Phase 4
12	Increment 3 Demo — QR Check-in & Certificates	Phase 5
13	Increment 4 Demo — Analytics Dashboard & Feedback	Phase 6
14	Testing Strategy & Traceability Matrix	Phase 7
15	Deployment Architecture (live URL)	Phase 8
16	Lessons Learned & Future Work	9.3

9.3 Lessons Learned & Reflection Points

- Designing repository interfaces (ISP) before writing any Supabase-specific code in Increment 1 paid off directly: Increments 2–4 added new services without ever modifying `EventService` or `RegistrationService`.
- RLS-first authorization (Section 0.4) meant that even features not yet covered by backend authorization checks were still protected at the database layer during early development — a safety net during active coding.
- Freezing each increment's mini-requirements (Sections 3.2, 4.2, 5.2, 6.2) before implementation directly mitigated the scope-creep risk identified in the Risk Register (Section 0.7).

- The Incremental model's core claim — that each increment is 'a meaningful addition to the system that can be integrated, tested, and delivered' rather than unfinished code — was validated in practice: after Increment 1 alone, a demonstrable, gradeable, end-to-end workflow already existed.

9.4 Closing Statement

CampusEvents demonstrates a complete software engineering lifecycle: a justified process-model choice, traceable requirements, an architecture and data model designed up front to anticipate all four increments, SOLID-compliant code reviewed against explicit checklists per increment, a requirements-to-test traceability matrix, and a real, free-tier deployment. Every artifact in this document — from the feasibility study's schedule table to the final increment's Definition of Done — is cross-referenced, so the project can be evaluated as a coherent engineering process rather than as isolated deliverables.

END OF DOCUMENT